SQL Interview Questions with Answers

1. SQL Fundamentals

1. What is SQL?

SQL (Structured Query Language) is a standard programming language used for managing and manipulating relational databases. It allows users to create, read, update, and delete data, as well as define and manage database structures.

2. What are the different types of SQL commands?

The different types of SQL commands are:

- DDL (Data Definition Language): Used to define and modify database structure (CREATE, ALTER, DROP)
- DML (Data Manipulation Language): Used to manipulate data within the database (INSERT, UPDATE, DELETE)
- DQL (Data Query Language): Used to retrieve data from the database (SELECT)
- DCL (Data Control Language): Used to control access to data (GRANT, REVOKE)
- TCL (Transaction Control Language): Used to manage transactions (COMMIT, ROLLBACK)

4. What is a database?

A database is an organized collection of structured information, or data, typically stored electronically in a computer system.

5. What is a table?

A table is a collection of related data organized in rows and columns within a database.

6. What is a row and a column?

A row represents a single record or entry in a table, while a column represents a specific attribute or field within the table.

7. What is a schema?

A schema is a logical container for database objects, such as tables, views, and indexes. It helps organize and manage the structure of the database.

8. What is the difference between a database and a schema?

A database is the overall container for all data and structures, while a schema is a logical division within the database that groups related objects together.

9. What is a primary key?

A primary key is a unique identifier for each record in a table. It must be unique and not null.

10. What is a foreign key?

A foreign key is a column or a set of columns in a table that references the primary key of another table. It establishes a relationship between the two tables.

11. What is a candidate key?

A candidate key is a column or a set of columns that can uniquely identify each row in a table. It must be unique and not null.

12. What is a super key?

A super key is a set of one or more columns that can uniquely identify each row in a table. It may contain additional columns beyond those necessary for uniqueness.

13. What is an alternate key?

An alternate key is a candidate key that is not chosen as the primary key. It can also uniquely identify each row in the table.

14. What is a composite key?

A composite key is a primary key that consists of two or more columns combined to uniquely identify each row in a table.

15. What is a surrogate key?

A surrogate key is an artificial or synthetic key that is used as a unique identifier for a record in a table. It is typically an auto-incremented integer or a UUID, and it has no business meaning.

16. What is a natural key?

A natural key is a key that has a real-world meaning and is derived from the data itself. It is used to uniquely identify records based on existing attributes, such as a Social Security Number or an email address.

17. What is the difference between `NULL`, `0`, and an empty string?

- `NULL` represents the absence of a value or unknown data. It indicates that the value is missing or not applicable.
- `0` is a numeric value that represents zero. It is a valid number and can be used in calculations.
- An empty string (`''`) is a string with no characters. It is a valid value and can be used in string operations, but it is not the same as `NULL` or `0`.

18. What is the difference between `WHERE` and `HAVING`?

- `WHERE` is used to filter rows before any grouping or aggregation takes place. It applies to individual rows in the table.
- `HAVING` is used to filter groups after aggregation has been performed. It applies to the results of a `GROUP BY` clause and is used to filter aggregated data.

19. What is the difference between `DELETE`, `TRUNCATE`, and `DROP`?

- `DELETE` is a DML command that removes specific rows from a table based on a condition. It can be rolled back if used within a transaction.
- `TRUNCATE` is a DDL command that removes all rows from a table, but it does not log individual row deletions. It is faster than `DELETE` but cannot be rolled back in some databases.
- `DROP` is a DDL command that removes an entire table or database, including its structure and data. It cannot be rolled back and is irreversible.

20. What is the difference between `UNION` and `UNION ALL`?

- `UNION` combines the results of two or more `SELECT` statements and removes duplicate rows from the final result set.
- `UNION ALL` combines results and retains duplicates; it's faster because it does not perform the duplicate-elimination step.

2. SQL Joins

21. What is a JOIN?

A `JOIN` combines rows from two or more tables based on a related column between them, producing a result set that includes columns from each table.

22. What are the different types of JOINs?

> Common JOINs: `INNER JOIN`, `LEFT (OUTER) JOIN`, `RIGHT (OUTER) JOIN`, `FULL (OUTER) JOIN`, `CROSS JOIN`, and `SELF JOIN`.

23. Explain INNER JOIN.

> `INNER JOIN` returns only rows where there is a match in both joined tables according to the join condition.

24. Explain LEFT JOIN.

> `LEFT JOIN` (or `LEFT OUTER JOIN`) returns all rows from the left table and matching rows from the right table; unmatched right-side columns are `NULL`.

25. Explain RIGHT JOIN.

> `RIGHT JOIN` returns all rows from the right table and matching rows from the left; unmatched left-side columns are `NULL`.

26. Explain FULL OUTER JOIN.

> `FULL OUTER JOIN` returns rows when there is a match in one of the tables; unmatched columns from either side are `NULL`.

27. What is a CROSS JOIN?

`CROSS JOIN` returns the Cartesian product of the two tables (every combination of rows).

28. What is a SELF JOIN?

A `SELF JOIN` joins a table to itself, useful for comparing rows within the same table (use aliases to distinguish sides).

29. What is the difference between INNER JOIN and LEFT JOIN?

`INNER JOIN` returns only matching rows from both tables; `LEFT JOIN` returns all left-table rows plus matches (or `NULL`) from the right.

30. What happens when there is no matching record in a LEFT JOIN?

The result includes the row from the left table with `NULL` values for the columns from the right table.

31. What is a Cartesian product?

The Cartesian product is the result of a `CROSS JOIN`: every row from table A paired with every row from table B; size = rows(A) * rows(B).

32. How can you identify duplicate records after a JOIN?

Use `GROUP BY` with `COUNT(*) > 1` or use `ROW_NUMBER()` window function partitioned by the join-key(s) to find duplicates.

33. What is the difference between JOIN conditions and WHERE conditions?

JOIN conditions determine how rows from tables are matched; `WHERE` filters the resulting rows. In `INNER JOIN` they often behave similarly, but for `OUTER JOIN` the `WHERE` can effectively convert it to an `INNER JOIN` if it filters on the outer table's columns.

34. Can we JOIN tables without a foreign key?

Yes. SQL does not require declared foreign keys to perform joins; you can join on any matching column(s).

35. How does SQL execute multiple JOINs?

The optimizer chooses a join order and join algorithms (nested loop, hash, merge) based on statistics and cost; execution proceeds according to the query plan.

3. SQL Constraints

36. What are SQL constraints?

> Constraints are rules applied to table columns to enforce data integrity (e.g., `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`).

37. What is a `NOT NULL` constraint?

> `NOT NULL` ensures a column cannot store `NULL` values.

38. What is a `UNIQUE` constraint?

> `UNIQUE` ensures all values in a column or group of columns are distinct.

39. What is a `CHECK` constraint?

> `CHECK` enforces that values in a column satisfy a boolean expression (e.g., `age >= 0`).

40. What is a `DEFAULT` constraint?

> `DEFAULT` provides a default value when an `INSERT` does not supply one.

41. What is a PRIMARY KEY constraint?

> A `PRIMARY KEY` uniquely identifies each row; it implies `UNIQUE` and `NOT NULL`.

42. What is a FOREIGN KEY constraint?

A `FOREIGN KEY` links a column (or columns) in one table to the primary key of another, enforcing referential integrity.

43. Can a table have multiple primary keys?

No — a table can have only one primary key, but that key can be composite (multiple columns).

44. Can a table have multiple UNIQUE constraints?

Yes. A table can have multiple `UNIQUE` constraints on different column(s) or combinations.

45. Can a foreign key contain NULL values?

Yes, unless the foreign-key column is declared `NOT NULL`; `NULL` means no reference is present.

46. What is referential integrity?

Referential integrity ensures relationships between tables remain consistent: foreign keys refer to existing primary-key values (or `NULL`).

47. What happens when a referenced record is deleted?

Behavior depends on foreign-key `ON DELETE` action: `RESTRICT/NO ACTION` prevents delete, `CASCADE` deletes dependent rows, `SET NULL` sets FK to `NULL`, `SET DEFAULT` sets to default.

48. What are `ON DELETE CASCADE` and `ON UPDATE CASCADE`?

They are foreign-key actions that automatically propagate deletes/updates from the parent row to child rows (delete or update matching child rows).

---

4. SQL Functions

49. What are SQL functions?

Functions are built-in or user-defined operations that accept inputs and return a single value (scalar) or a result set (table-valued functions).

50. What are aggregate functions?

Aggregates compute a single result from multiple input rows (e.g., `SUM`, `AVG`, `COUNT`, `MIN`, `MAX`).

51. What are scalar functions?

Scalar functions return a single value per input row (e.g., `UPPER()`, `LENGTH()`, `ABS()`).

52. Explain `COUNT()`.

`COUNT()` returns the number of input rows: `COUNT(*)` counts all rows, `COUNT(column)` counts non-`NULL` values in that column.

53. Difference between `COUNT(*)` and `COUNT(column)`.

`COUNT(*)` counts all rows including those with `NULL` in columns; `COUNT(column)` counts only rows where `column` is not `NULL`.

54. Explain `SUM()`, `AVG()`, `MIN()`, and `MAX()`.

`SUM()` adds numeric values; `AVG()` computes their average; `MIN()` returns the smallest value; `MAX()` returns the largest. They ignore `NULL`s.

55. How does `COUNT()` handle NULL values?

`COUNT(column)` ignores `NULL`s; `COUNT(*)` counts rows regardless of `NULL`.

56. What is `COALESCE()`?

`COALESCE(a,b,...)` returns the first non-`NULL` expression from its arguments.

57. What is `NULLIF()`?

`NULLIF(a,b)` returns `NULL` if `a = b`, otherwise returns `a`.

58. What are string functions?

Functions for manipulating text: `CONCAT`, `SUBSTRING`, `TRIM`, `UPPER`, `LOWER`, `REPLACE`, `LENGTH`, etc.

59. What are date/time functions?

Functions to work with dates/times: `NOW()`, `CURRENT_DATE`, `DATEADD`/`INTERVAL`, `DATEDIFF`, `EXTRACT`, `TO_DATE`, `TO_CHAR`.

60. What are mathematical functions?

Numeric functions: `ABS`, `CEIL`/`CEILING`, `FLOOR`, `ROUND`, `POWER`, `SQRT`, `MOD`.

5. GROUP BY & HAVING

61. What is `GROUP BY`?

> `GROUP BY` groups rows that share a property so aggregate functions can be applied per group.

62. Why do we use `GROUP BY`?

> To aggregate data (sum, count, avg) per category or grouping key.

63. What is the difference between `GROUP BY` and `ORDER BY`?

> `GROUP BY` groups rows for aggregation; `ORDER BY` sorts the final result set.

64. Can we use aggregate functions without `GROUP BY`?

> Yes — aggregates can operate over the whole result set without `GROUP BY` (single-group aggregate).

65. Why can't we normally use a non-aggregated column in SELECT with GROUP BY?

> Non-aggregated columns must be functionally dependent on the `GROUP BY` columns; otherwise the result would be ambiguous.

66. What is `HAVING`?

> `HAVING` filters groups after aggregation, similar to `WHERE` but for grouped results.

67. Why is `HAVING` used instead of `WHERE` for aggregate conditions?

> Because `WHERE` filters rows before aggregation and cannot use aggregate results; `HAVING` filters after aggregation when aggregates exist.

68. Can we use both WHERE and HAVING in the same query?

> Yes. Use `WHERE` to filter rows before grouping and `HAVING` to filter aggregated groups.

69. What is the execution order of WHERE, GROUP BY, HAVING, and SELECT?

> Logical order: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`.

6. Subqueries

70. What is a subquery?

> A subquery is a query nested inside another query (in `SELECT`, `FROM`, `WHERE`, etc.) whose result is used by the outer query.

71. What are the different types of subqueries?

Types: scalar (single value), row, column, table subqueries; correlated and non-correlated subqueries.

72. What is a scalar subquery?

A scalar subquery returns a single value (one row, one column) to be used in expressions.

73. What is a correlated subquery?

A correlated subquery references columns from the outer query and is evaluated per outer row.

74. What is a non-correlated subquery?

A non-correlated subquery is independent of the outer query and can be executed once.

75. What is the difference between a JOIN and a subquery?

`JOIN` combines tables into a single result set; subqueries compute values or intermediate sets used by the outer query. `JOIN`s are often more efficient for returning columns from multiple tables.

76. When should you use a JOIN instead of a subquery?

Use `JOIN` when you need columns from both tables in the result or when performance favors set-based joins; use subqueries for scalar lookups or existence checks.

77. What is the difference between `IN` and `EXISTS`?

`IN` tests membership against a list/result set; `EXISTS` checks if a correlated subquery returns any row. With large result sets, `EXISTS` often performs better.

78. What is the difference between `NOT IN` and `NOT EXISTS`?

`NOT IN` can behave unexpectedly if the subquery returns `NULL` (it will return no rows); `NOT EXISTS` handles `NULL` safely and is generally preferred for correlated absence checks.

79. What problems can occur with `NOT IN` when NULL exists?

If the subquery returns any `NULL`, `NOT IN` comparisons produce unknown results and may return no rows; this makes `NOT IN` unsafe unless the subquery excludes `NULL`s.

7. SQL Set Operations

80. What are set operators in SQL?

> Set operators combine result sets: `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`/`MINUS`.

81. Explain `UNION`.

> `UNION` returns distinct rows present in either result set.

82. Explain `UNION ALL`.

> `UNION ALL` returns all rows from both result sets, including duplicates.

83. Explain `INTERSECT`.

> `INTERSECT` returns rows common to both result sets.

84. Explain `EXCEPT`.

> `EXCEPT` (or `MINUS`) returns rows from the first result set that are not in the second.

85. Difference between UNION and UNION ALL.

> `UNION` removes duplicates; `UNION ALL` keeps duplicates and is faster.

86. What conditions must be satisfied when using UNION?

Each `SELECT` must have the same number of columns and compatible data types in corresponding positions.

87. Which is generally faster: UNION or UNION ALL, and why?

`UNION ALL` is faster because it does not need to sort/deduplicate results.

---

8. SQL Views

88. What is a VIEW?

A `VIEW` is a virtual table defined by a `SELECT` query that presents data from one or more tables.

89. Why do we use views?

Views simplify complex queries, encapsulate logic, provide security by limiting columns/rows, and present consistent abstractions.

90. What is the difference between a table and a view?

A table stores data physically; a view is a saved query (virtual) that presents data from underlying tables.

91. What is a materialized view?

A materialized view stores the result of the view physically and must be refreshed to reflect underlying changes.

92. Difference between a view and materialized view.

Views are virtual and always reflect current data; materialized views store data and offer faster reads at the cost of maintenance/refresh.

93. Can we insert/update/delete through a view?

Sometimes: updatable views allow DML when they map unambiguously to base tables; complex views, aggregates, or joins often are not updatable.

94. What are the advantages of views?

Abstraction, simplified queries, security, reusability, and consistent interfaces.

95. What are the disadvantages of views?

Performance overhead for complex views, potential inability to index (unless materialized), and maintenance complexity.

96. When should you use a materialized view?

Use when queries are expensive and data can tolerate some staleness; materialized views speed repeated reads.

9. Indexes

97. What is an index?

An index is a database structure that speeds up data retrieval for queries by providing quick lookup paths.

98. Why do we need indexes?

To reduce query latency by avoiding full table scans for selective queries.

99. How does an index improve query performance?

Indexes allow the database to find rows by looking up keys instead of scanning all rows, using structures like B-trees or hashes.

100. What are the disadvantages of indexes?

Increased storage, slower `INSERT`/`UPDATE`/`DELETE` (index maintenance), and possible poor performance if misused.

101. What is a clustered index?

A clustered index defines the physical order of rows in a table (one per table in many DBMSs); the table is stored in index order.

102. What is a non-clustered index?

A non-clustered index maintains a separate structure mapping key values to row locations without altering physical row order.

103. What is a B-tree index?

B-tree is a balanced-tree index structure suitable for range scans and ordered lookup; it's the most common index type.

104. What is a hash index?

A hash index uses a hash table for equality lookups; efficient for `=` but not for range queries.

105. What is a composite index?

An index on multiple columns; used when queries filter or sort by combinations of columns.

106. What is a partial index?

A partial index indexes only a subset of rows that satisfy a predicate, reducing index size and improving selectivity for that subset.

107. What is a unique index?

A unique index enforces uniqueness of the indexed column(s).

108. What is a covering index?

A covering index contains all columns needed by a query so the DBMS can satisfy the query from the index alone.

109. What is an index scan?

An index scan reads entries from an index to locate rows; it may be more efficient than a full table scan when selective.

110. What is a sequential scan?

A sequential (table) scan reads all table rows; chosen when table is small or index is not selective.

111. When will PostgreSQL choose a sequential scan instead of an index scan?

When the planner estimates that scanning the whole table is cheaper (low selectivity, small table, outdated statistics, or when many rows are requested).

112. What is index selectivity?

Selectivity measures how well an index discriminates rows; high selectivity (many distinct values) is better for index usage.

113. What is index cardinality?

Cardinality is the number of distinct values in the indexed column; related to selectivity.

114. What is the leftmost-prefix rule for composite indexes?

For composite indexes, the leading (leftmost) column(s) must be used in the query predicates/order to utilize the index efficiently.

115. Why can too many indexes hurt performance?

Each additional index increases storage and slows write operations due to index maintenance.

116. Can an index slow down INSERT/UPDATE/DELETE?

Yes — DML operations must update indexes, adding overhead.

117. How do you identify unused indexes?

Use DBMS-specific stats/views (e.g., PostgreSQL's `pg_stat_user_indexes` and `pg_stat_all_indexes`) and query plans to find indexes with low usage and high maintenance cost.

10. Transactions

118. What is a transaction?

A transaction is a sequence of one or more SQL statements executed as a single logical unit with atomicity: all succeed or all fail.

119. What are ACID properties?

ACID: Atomicity, Consistency, Isolation, Durability — guarantees for reliable transactions.

120. Explain Atomicity.

Atomicity ensures all operations in a transaction complete successfully or none do (rollback on failure).

121. Explain Consistency.

Consistency ensures a transaction brings the database from one valid state to another, preserving integrity constraints.

122. Explain Isolation.

Isolation controls how concurrently executing transactions affect each other; higher isolation reduces interference.

123. Explain Durability.

Durability ensures committed transactions persist even after crashes (written to stable storage).

124. What is `COMMIT`?

`COMMIT` makes all changes in the current transaction permanent.

125. What is `ROLLBACK`?

`ROLLBACK` undoes all changes made in the current transaction.

126. What is `SAVEPOINT`?

`SAVEPOINT` creates a named point inside a transaction to which you can roll back without aborting the entire transaction.

127. What happens if a transaction fails?

The DBMS rolls back the transaction (or the application issues `ROLLBACK`) to preserve consistency.

128. What is transaction isolation?

Isolation defines visibility rules for concurrent transactions and controls phenomena like dirty reads, non-repeatable reads, and phantom reads.

129. What are isolation levels?

Standard levels: `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`.

130. Explain READ UNCOMMITTED.

`READ UNCOMMITTED` allows reading uncommitted changes (dirty reads); rarely used due to inconsistency.

131. Explain READ COMMITTED.

`READ COMMITTED` only reads committed data; a query in a transaction sees data committed before that statement began.

132. Explain REPEATABLE READ.

`REPEATABLE READ` ensures that repeated reads within a transaction return the same rows; prevents non-repeatable reads but may allow phantoms depending on DBMS.

133. Explain SERIALIZABLE.

`SERIALIZABLE` provides the strictest isolation, guaranteeing results equivalent to some serial execution of transactions.

134. What is a dirty read?

A dirty read occurs when a transaction reads uncommitted changes made by another transaction.

135. What is a non-repeatable read?

A non-repeatable read occurs when a transaction re-reads a row and finds that another committed transaction has modified it.

136. What is a phantom read?

A phantom read occurs when a transaction re-executes a query and finds new rows inserted by other transactions that match the query predicate.

137. What is a lost update?

A lost update happens when two transactions read the same row and both update it, with one update overwriting the other without awareness.

11. Normalization

138. What is database normalization?

Normalization is organizing data to reduce redundancy and improve integrity by decomposing tables according to normal forms.

139. Why do we normalize databases?

To remove redundancy, prevent update anomalies, and ensure data integrity.

140. What is 1NF?

First Normal Form: atomic (indivisible) values and each field contains only one value; no repeating groups.

141. What is 2NF?

Second Normal Form: in 1NF and every non-key attribute is fully functionally dependent on the whole primary key (no partial dependencies).

142. What is 3NF?

Third Normal Form: in 2NF and no transitive dependencies (non-key attributes depend only on the key).

143. What is BCNF?

Boyce–Codd Normal Form: stronger than 3NF; every determinant is a candidate key.

144. What is 4NF?

Fourth Normal Form: no multi-valued dependencies other than a candidate key.

145. What is 5NF?

Fifth Normal Form: decomposed to eliminate redundancy caused by join dependencies; rarely needed.

146. What is denormalization?

Denormalization intentionally introduces redundancy to improve read performance at the cost of update complexity.

147. What are the advantages of normalization?

Reduced redundancy, easier updates, improved consistency, and smaller storage.

148. What are the disadvantages of excessive normalization?

More joins, potentially slower read queries, complexity in queries and reporting.

149. When should we denormalize a database?

When read performance is critical and after profiling shows joins are the bottleneck; use carefully with mechanisms to maintain consistency.

150. What is functional dependency?

A relationship where one attribute (or set) determines another attribute (A → B means value of A uniquely determines B).

151. What is partial dependency?

Partial dependency exists when a non-key attribute depends on part of a composite primary key.

152. What is transitive dependency?

Transitive dependency: A → B and B → C implies A → C; if C depends on non-key attribute B, it's a transitive dependency.

12. Window Functions

153. What is a window function?

A window function performs calculations across a set of table rows related to the current row without collapsing the rows into a single output row.

154. How is a window function different from GROUP BY?

`GROUP BY` aggregates rows into single summary rows per group; window functions compute values across partitions while preserving individual rows.

155. What is `PARTITION BY`?

`PARTITION BY` divides rows into partitions (groups) over which the window function operates.

156. What is `ORDER BY` inside a window function?

`ORDER BY` defines the order of rows within each partition, important for ranking and cumulative calculations.

157. Difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`.

`ROW_NUMBER()` gives a unique sequential number per row; `RANK()` gives tied rows the same rank with gaps after ties; `DENSE_RANK()` gives tied rows same rank without gaps.

158. What are `LEAD()` and `LAG()`?

`LEAD()` and `LAG()` access subsequent or prior row values in the partition, useful for comparisons and trend calculations.

159. What is a running total?

A running total (cumulative sum) is calculated by a window function like `SUM(value) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

160. How do you calculate a cumulative sum?

Use `SUM(column) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

161. How do you find the first and last record of each group?

Use `ROW_NUMBER()` ordered appropriately partitioned by the group and then filter `WHERE row_num = 1` (or use `FIRST_VALUE()`/`LAST_VALUE()`).

162. Can window functions be used in WHERE?

No — window functions are computed after `WHERE`; use a subquery/CTE to filter on window results.

163. Why do we use a subquery/CTE when filtering window-function results?

Because window functions are evaluated after `WHERE` and `GROUP BY`, so a subquery/CTE allows filtering by their computed results.

13. CTE

164. What is a CTE?

CTE (Common Table Expression) is a named temporary result set defined using `WITH` that can be referenced within a query.

165. Why do we use CTEs?

For readability, modular query building, recursion, and to avoid repeating complex subqueries.

166. What is the difference between CTE and subquery?

CTEs are named and can be easier to read and reference multiple times; subqueries are anonymous and nested in expressions.

167. What is a recursive CTE?

A recursive CTE refers to itself to perform iterative operations (e.g., traversing hierarchies, transitive closures).

168. Where are recursive CTEs useful?

Hierarchies (org charts), graph traversal, bill-of-materials, and tasks requiring iterative expansion.

169. Can a CTE improve query performance?

Sometimes — CTEs can help the optimizer when used appropriately, but some DBMS treat them as optimization fences; test performance.

170. What is the difference between a CTE and temporary table?

Temporary tables persist for the session and can be indexed; CTEs are in-query and generally not materialized (implementation-dependent).

171. What are the advantages and disadvantages of CTEs?

Advantages: readability, modularity, recursion. Disadvantages: possible performance implications (materialization) in some DBMSs.

14. Stored Procedures & Functions

172. What is a stored procedure?

A stored procedure is a precompiled collection of SQL statements and optional control-flow logic stored in the database and callable by name.

173. What is a stored function?

A stored function returns a value and can be used in SQL expressions; procedures are invoked with `CALL` or similar.

174. Difference between procedure and function.

Functions return a value and can be used in queries; procedures perform actions and may not return a value (DBMS-specific differences exist).

175. What are the advantages of stored procedures?

Encapsulation, reduced client-server traffic, centralized business logic, potential performance gains, and security control.

176. What are the disadvantages?

Portability issues, harder version control, complexity in debugging, and potential overuse leading to business logic in DB.

177. What are input and output parameters?

Parameters passed to procedures/functions (`IN`, `OUT`, `INOUT`) to supply inputs and return outputs.

178. Can a function return a table?

Yes — many DBMSs support table-valued functions that return a set of rows.

179. What is exception handling in stored procedures?

Mechanisms to catch and handle runtime errors within procedures (e.g., `EXCEPTION` blocks in PL/pgSQL, `TRY/CATCH` in T-SQL).

180. What is dynamic SQL?

Dynamic SQL is SQL constructed and executed at runtime (e.g., building query strings and executing them), allowing flexible queries.

181. What are the risks of dynamic SQL?

SQL injection, complexity, harder optimization, and potential performance/security issues if not parameterized properly.

---

15. Database Design

182. What is database design?

Database design is the process of modeling entities, attributes, relationships, and physical structures to meet application requirements.

183. What factors should be considered while designing a database?

Data access patterns, normalization, scalability, indexing, partitioning, backup/restore, security, and future schema evolution.

184. How do you identify entities?

Entities map to real-world objects or concepts with attributes; identify nouns in requirements and domain models.

185. What are attributes?

Attributes are properties or fields that describe an entity (columns in a table).

186. What are relationships?

Relationships describe how entities are associated (one-to-one, one-to-many, many-to-many).

187. What is an ER diagram?

Entity-Relationship diagram visually represents entities, attributes, and their relationships.

188. What is cardinality?

Cardinality specifies the number of relationships between entities (e.g., one-to-many).

189. Explain one-to-one relationship.

Each row in table A relates to at most one row in table B and vice versa.

190. Explain one-to-many relationship.

One row in table A relates to many rows in table B; implemented using a foreign key in the many-side table.

191. Explain many-to-many relationship.

Many rows in A relate to many rows in B and are implemented using a junction (bridge) table.

192. How do you implement many-to-many relationships?

Use a junction table that contains foreign keys to both related tables and optionally additional attributes.

193. What is a junction/bridge table?

A table that resolves a many-to-many relationship by storing pairs of foreign keys referencing the two tables.

194. How do you choose a primary key?

Choose a stable, unique attribute (natural key) or use a surrogate key (auto-increment/UUID) when no stable natural key exists.

195. Natural key vs surrogate key?

Natural key has real-world meaning; surrogate key is artificial and solely identifies rows (e.g., `id` column). Surrogates simplify joins and evolution.

196. How do you decide whether to normalize or denormalize?

Base decision on read/write patterns, performance requirements, and maintainability—normalize for integrity, denormalize for performance where needed.

197. How do you design an audit table?

Capture who/when/what changed, include reference to the primary key, operation type, before/after values, timestamp, and user/context.

198. How do you design a soft-delete mechanism?

Add an `is_deleted` boolean or `deleted_at` timestamp; ensure queries filter out deleted rows and consider indexing the flag.

199. How do you handle historical data?

Use temporal tables, history tables, partitioning, or archive to separate cold data; choose retention and access patterns.

200. How do you design a multi-tenant database?

Options: separate databases, separate schemas, or shared schema with tenant identifier column; choose based on isolation, scale, and cost.

16. SQL Performance & Optimization

201. What is query optimization?

Query optimization is selecting an efficient execution plan using statistics, indexes, and rewrite rules to minimize cost.

202. How do you optimize a slow SQL query?

Analyze the `EXPLAIN` plan, add/adjust indexes, rewrite queries for set-based operations, update statistics, and consider partitioning or caching.

203. What is an execution plan?

A plan describes the steps the DBMS will take to execute a query, including join order, access methods, and estimated costs.

204. What is `EXPLAIN`?

`EXPLAIN` shows the DBMS's planned execution strategy for a query (without running it, or with runtime stats in some DBMSs).

205. What is `EXPLAIN ANALYZE`?

`EXPLAIN ANALYZE` runs the query and returns the actual runtime statistics alongside the plan for accurate diagnostics.

206. What is query cost?

Query cost is an estimated measure (CPU, I/O, memory) the planner uses to compare execution plans.

207. What is a sequential scan?

Reading the table row-by-row; used when it's cheaper than using an index.

208. What is an index scan?

Using an index to lookup rows matching predicates instead of scanning the whole table.

209. What is a bitmap heap scan?

A two-step scan where the planner builds a bitmap of matching row locations from one or more indexes and then fetches rows from the heap efficiently.

210. What is nested loop join?

A join algorithm that iterates rows from one table and for each probes the other (efficient for small outer sets or indexed inner tables).

211. What is hash join?

A join algorithm that builds a hash table on the smaller input and probes it with the larger input; efficient for large, unsorted joins.

212. What is merge join?

A join algorithm that requires both inputs sorted on the join key and merges them in linear time; efficient for range-ordered joins.

213. How do you identify the bottleneck in a query?

Use `EXPLAIN ANALYZE`, check I/O, CPU, memory, locks, and look at actual vs estimated row counts to find mismatches.

214. Why can SELECT * be problematic?

`SELECT *` returns unnecessary columns, increases I/O, prevents index-only scans and can break with schema changes.

215. Why should functions on indexed columns generally be avoided?

Applying functions can prevent the use of indexes unless functional indexes exist, reducing index effectiveness.

216. What is predicate pushdown?

Moving filter predicates as close to the data source as possible (e.g., into scans or storage engines) to reduce processed rows.

217. What is partition pruning?

Eliminating partitions from consideration at query time based on predicates, reducing scanned data.

218. What is query parallelism?

Executing parts of a query across multiple CPU workers to improve throughput for large operations.

219. How can statistics affect query performance?

Accurate statistics help the planner choose the best plan; stale/missing stats can lead to poor plans.

220. What is table/index bloat?

Bloat is wasted space from MVCC and updates/deletes; it increases IO and may require vacuuming/reindexing.

---

17. Advanced SQL Concepts

221. What is recursive SQL?

SQL that uses recursion (often via recursive CTEs) to iterate over hierarchical or self-referential data.

222. What are lateral joins?

`LATERAL` allows a subquery in the `FROM` clause to reference columns from preceding tables in the same `FROM` list.

223. What is a correlated query?

A subquery that references columns from the outer query and executes per outer row.

224. What is dynamic SQL?

SQL constructed and executed at runtime (see earlier); allows flexible table/column names and predicates.

225. What is pivoting?

Pivoting rotates rows into columns to present aggregated data in a cross-tab format.

226. What is unpivoting?

Unpivoting converts columns into rows to normalize denormalized columnar data.

227. What is an UPSERT?

An UPSERT inserts a row or updates it if it already exists (e.g., `INSERT ... ON CONFLICT DO UPDATE` in PostgreSQL or `MERGE`).

228. What is `MERGE`?

`MERGE` performs conditional insert/update/delete in a single statement based on matching between source and target.

229. What is an idempotent SQL operation?

An operation that can be applied multiple times without changing the result beyond the initial application (safe for retries).

230. What is optimistic locking?

A concurrency control strategy that detects conflicts at commit time (version/timestamp checks) rather than locking rows.

231. What is pessimistic locking?

A strategy that acquires locks on rows/tables to prevent concurrent modifications while the transaction runs.

232. What is a deadlock?

A situation where two or more transactions wait indefinitely for resources held by each other.

233. How does a database detect deadlocks?

The DBMS detects cycles in the wait-for graph and aborts one transaction to break the cycle.

234. How can deadlocks be prevented?

Keep transactions short, access resources in a consistent order, use lower isolation or retry logic, and acquire locks deliberately.

235. What is row-level locking?

Locking individual rows to allow concurrent access to other rows in the table.

236. What is table-level locking?

Locking entire tables which serializes access and reduces concurrency.

237. What is MVCC?

Multi-Version Concurrency Control maintains multiple versions of rows so readers see a snapshot without blocking writers.

238. Why is MVCC important?

MVCC improves concurrency by allowing readers and writers to proceed without mutual blocking and provides snapshot isolation semantics.

239. What is database concurrency?

Ability for multiple transactions to access and modify the database simultaneously while maintaining correctness.

240. What is connection pooling?

Reusing a set of open database connections to reduce overhead of establishing connections per request.

18. PostgreSQL-Specific Interview Questions

241. What is MVCC in PostgreSQL?

PostgreSQL's MVCC implementation stores multiple row versions (tuples) so readers can access a consistent snapshot while writers create new versions.

242. How does PostgreSQL handle concurrent transactions?

Using MVCC and snapshot isolation semantics, PostgreSQL allows concurrent reads/writes and resolves conflicts with locking or conflict detection.

243. What is VACUUM?

`VACUUM` reclaims space from dead tuples created by updates/deletes and updates visibility map/statistics.

244. What is VACUUM FULL?

`VACUUM FULL` rewrites the entire table to compact it and reclaim disk space but requires exclusive locks and is slower.

245. Difference between VACUUM and VACUUM FULL.

`VACUUM` marks dead space for reuse; `VACUUM FULL` rebuilds the table physically and can free OS-level disk space.

246. What is ANALYZE?

`ANALYZE` collects table statistics used by the planner to make informed optimization decisions.

247. What is autovacuum?

`autovacuum` automatically runs `VACUUM`/`ANALYZE` in the background to maintain table health.

248. What is table bloat?

Bloat is wasted space due to dead tuples and fragmentation from MVCC operations; it increases table size and degrades performance.

249. What is `pg_stat_activity`?

`pg_stat_activity` is a view that shows current database sessions and their activity (queries, state, backend pid).

250. How do you find currently running queries?

Query `pg_stat_activity` (or use `pg_stat_statements`) to inspect currently running queries and their durations.

251. How do you find long-running queries?

Filter `pg_stat_activity` by `state = 'active'` and by `now() - query_start` to identify long-running statements.

252. How do you terminate a PostgreSQL session?

Use `SELECT pg_terminate_backend(pid)` with the target `pid` from `pg_stat_activity` (requires proper permissions).

253. What is WAL?

Write-Ahead Log (WAL) records changes before they are applied to data files to ensure durability and support replication/recovery.

254. What is a checkpoint?

A checkpoint flushes dirty buffers to disk and records a point in WAL from which recovery can start.

255. What is PostgreSQL streaming replication?

Streaming replication streams WAL records from primary to standby servers to keep replicas up-to-date in near real-time.

256. What is logical replication?

Logical replication replicates data changes at a logical level (tables/rows) and can replicate subset of data and handle schema differences.

257. Physical vs logical replication.

Physical replication replicates binary WAL and entire cluster state; logical replication replicates logical changes per table and is more flexible.

258. What is partitioning in PostgreSQL?

Partitioning splits a large table into smaller child tables (partitions) based on a partition key to improve manageability and performance.

259. What are RANGE, LIST, and HASH partitions?

`RANGE` splits by value ranges, `LIST` by discrete lists of values, `HASH` by hash modulus for even distribution.

260. What is partition pruning?

Partition pruning eliminates irrelevant partitions during planning based on query predicates to reduce scanned data.

261. What is a default partition?

A default partition catches rows that don't match other partition constraints.

262. What happens when inserting a record that doesn't match a partition?

The insert fails unless a `DEFAULT` partition exists to accept unmatched rows.

263. What are sequences?

Sequences are objects that generate unique numeric values, often used for auto-incrementing primary keys.

264. Difference between `SERIAL` and `IDENTITY`.

`SERIAL` is a legacy PostgreSQL shorthand that creates a sequence and sets default; `IDENTITY` is SQL standard for auto-generated values with more defined semantics.

265. What is JSONB?

`JSONB` is a binary JSON storage format in PostgreSQL that stores JSON in a parsed, indexable form.

266. JSON vs JSONB?

`JSON` stores raw text, preserving formatting; `JSONB` stores binary parsed JSON, faster for querying and indexing but may reorder keys.

267. How do you index JSONB?

Use GIN indexes (e.g., `CREATE INDEX ON table USING gin (jsonb_column)`), optionally with jsonb_path_ops for certain operations.

268. What are GIN and GiST indexes?

GIN (Generalized Inverted Index) is good for indexing array/JSON keys; GiST (Generalized Search Tree) supports range and similarity indexing; both are extensible.

269. B-tree vs GIN vs GiST?

B-tree is for ordered scalar data and range/equality; GIN is for multi-key lookups (arrays, JSON); GiST supports complex data types and nearest-neighbor searches.

270. What is an extension in PostgreSQL?

An extension is a packaged add-on providing extra types, functions, or tools (e.g., `postgis`, `pg_trgm`) that can be installed into a database.

19. Senior-Level Interview Questions

These are the questions I'd prioritize for a Senior Data Engineer interview:

1. How would you design a database for 100 million+ records?

Use partitioning, appropriate indexing, compression, vertical/horizontal sharding if needed, careful schema design, and plan for archiving and maintenance.

2. How would you optimize a query taking 30 seconds?

Analyze `EXPLAIN ANALYZE`, identify slow operators, add indexes or rewrite query, update stats, consider partitioning, caching or materialized views.

3. How would you identify the root cause of database latency?

Monitor metrics (CPU, I/O, locks), inspect query plans, check slow queries, connection pool, background tasks, and system-level bottlenecks.

4. How do you decide which columns should be indexed?

Index columns used in `WHERE` predicates, join keys, and `ORDER BY`/`GROUP BY` when selective; balance with write overhead.

5. When would you avoid creating an index?

Avoid indexing low-selectivity columns, write-heavy tables where maintenance cost outweighs read benefits, or rarely-used columns.

6. How would you design partitioning for a very large table?

Choose partition key based on query patterns (time ranges, tenant id), use appropriate partition type, and ensure partitions are manageable in size.

7. How would you choose between RANGE, LIST, and HASH partitioning?

Use `RANGE` for time-based queries, `LIST` for discrete sets (regions/tenants), `HASH` for even distribution when queries are evenly spread.

8. How would you handle data retention?

Implement automated archiving and deletion policies, partition-based retention for efficient drop, and legal/compliance requirements.

9. How would you archive historical data?

Move to cheaper storage, archive tables/databases, or export to data lake; keep summarized aggregates in primary DB if needed.

10. How would you handle concurrent updates?

Use transactions with appropriate isolation, optimistic locking/versioning for high concurrency, and short transactions to reduce contention.

11. How would you prevent duplicate records?

Enforce `UNIQUE` constraints, use deduplication during ingestion, idempotent loading (upserts), and application-level checks.

12. How would you design an idempotent data-loading process?

Use keys for deduplication, upserts with `ON CONFLICT`, checkpoints/cursors, and idempotent job semantics (replay-safe operations).

13. How would you design an audit mechanism?

Use triggers or application-level logging to write immutable audit records with who/when/what, or use event sourcing for full history.

14. How would you handle schema changes in production?

Use backward/forward-compatible migrations, deploy in phases (add columns, backfill, switch reads, drop old columns), and test carefully.

15. How would you troubleshoot deadlocks?

Capture deadlock traces, analyze involved transactions and resource order, fix by reordering access or reducing transaction scope, add retries.

16. How would you troubleshoot connection exhaustion?

Check connection pool settings, limit concurrent clients, use pooling proxies (PgBouncer), and ensure efficient connection reuse.

17. How would you troubleshoot high database CPU?

Identify expensive queries via `pg_stat_statements`, optimize queries, check for bad plans, and consider adding indexes or scaling the hardware.

18. How would you troubleshoot high database I/O?

Identify IO-heavy queries/operations, optimize queries, reduce bloat, adjust checkpointing, use faster storage or caching.

19. How would you troubleshoot high query latency during load testing?

Use profiling, identify slow queries, compare plans under load, inspect locks/contention, tune configuration and scale resources.

20. How would you design an OLTP database?

Normalize for consistency, favor small transactions, appropriate indexing, consider partitioning, and ensure low-latency writes.

21. How would you design an OLAP database?

Denormalize or use star/snowflake schema, use columnar storage/warehouse, pre-aggregate, and optimize for large analytical queries.

22. OLTP vs OLAP?

OLTP handles transactional workloads with many small reads/writes; OLAP handles analytical workloads with large, complex reads and aggregations.

23. Star schema vs snowflake schema?

Star schema: denormalized dimension tables; Snowflake: normalized dimensions. Star is simpler and faster for queries; snowflake reduces redundancy.

24. What is CDC?

Change Data Capture records and streams row-level changes (inserts/updates/deletes) for downstream consumers.

25. What is Change Data Capture used for?

For replication, ETL/ELT, analytics, synchronization, and event-driven architectures.

26. How would you implement incremental loading?

Use watermark columns (timestamp or increasing id), CDC, or delta detection to load only changed rows.

27. How would you handle late-arriving data?

Implement windowed reprocessing, backfill processes, or use event-time processing with corrections and idempotent updates.

28. How would you handle duplicate events?

De-duplicate using unique keys, dedupe buffers, idempotent writes, or transactional dedupe in the sink.

29. How would you maintain data consistency between source and target?

Use checksums, row counts, idempotent loading, transactional replication, and reconciliation jobs to detect drift.

30. How would you design a database supporting millions of transactions per day?

Scale horizontally/vertically, use partitioning/sharding, tune connection pooling, optimize hot paths, and use fast storage and caching.

2. What are the different types of SQL commands?

The different types of SQL commands are:

- DDL (Data Definition Language): Used to define and modify database structure (CREATE, ALTER, DROP)
- DML (Data Manipulation Language): Used to manipulate data within the database (INSERT, UPDATE, DELETE)
- DQL (Data Query Language): Used to retrieve data from the database (SELECT)
- DCL (Data Control Language): Used to control access to data (GRANT, REVOKE)
- TCL (Transaction Control Language): Used to manage transactions (COMMIT, ROLLBACK)

3. What is the difference between DDL, DML, DQL, DCL, and TCL?

- DDL (Data Definition Language) is used to define and modify database structure.
- DML (Data Manipulation Language) is used to manipulate data within the database.
- DQL (Data Query Language) is used to retrieve data from the database.
- DCL (Data Control Language) is used to control access to data.
- TCL (Transaction Control Language) is used to manage transactions.

4. What is a database?

A database is an organized collection of structured information, or data, typically stored electronically in a computer system.

5. What is a table?

A table is a collection of related data organized in rows and columns within a database.

6. What is a row and a column?

A row represents a single record or entry in a table, while a column represents a specific attribute or field within the table.

7. What is a schema?

A schema is a logical container for database objects, such as tables, views, and indexes. It helps organize and manage the structure of the database.

8. What is the difference between a database and a schema?

A database is the overall container for all data and structures, while a schema is a logical division within the database that groups related objects together.

9. What is a primary key?

A primary key is a unique identifier for each record in a table. It must be unique and not null.

10. What is a foreign key?

A foreign key is a column or a set of columns in a table that references the primary key of another table. It establishes a relationship between the two tables.

11. What is a candidate key?

A candidate key is a column or a set of columns that can uniquely identify each row in a table. It must be unique and not null.

12. What is a super key?

A super key is a set of one or more columns that can uniquely identify each row in a table. It may contain additional columns beyond those necessary for uniqueness.

13. What is an alternate key?

An alternate key is a candidate key that is not chosen as the primary key. It can also uniquely identify each row in the table.

14. What is a composite key?

A composite key is a primary key that consists of two or more columns combined to uniquely identify each row in a table.

15. What is a surrogate key?

A surrogate key is an artificial or synthetic key that is used as a unique identifier for a record in a table. It is typically an auto-incremented integer or a UUID, and it has no business meaning.

16. What is a natural key?

A natural key is a key that has a real-world meaning and is derived from the data itself. It is used to uniquely identify records based on existing attributes, such as a Social Security Number or an email address.

17. What is the difference between `NULL`, `0`, and an empty string?

- `NULL` represents the absence of a value or unknown data. It indicates that the value is missing or not applicable.
- `0` is a numeric value that represents zero. It is a valid number and can be used in calculations.
- An empty string (`''`) is a string with no characters. It is a valid value and can be used in string operations, but it is not the same as `NULL` or `0`.

18. What is the difference between `WHERE` and `HAVING`?

- `WHERE` is used to filter rows before any grouping or aggregation takes place. It applies to individual rows in the table.
- `HAVING` is used to filter groups after aggregation has been performed. It applies to the results of a `GROUP BY` clause and is used to filter aggregated data.

19. What is the difference between `DELETE`, `TRUNCATE`, and `DROP`?

- `DELETE` is a DML command that removes specific rows from a table based on a condition. It can be rolled back if used within a transaction.
- `TRUNCATE` is a DDL command that removes all rows from a table, but it does not log individual row deletions. It is faster than `DELETE` but cannot be rolled back in some databases.
- `DROP` is a DDL command that removes an entire table or database, including its structure and data. It cannot be rolled back and is irreversible.

20. What is the difference between `UNION` and `UNION ALL`?

- `UNION` combines the results of two or more SELECT statements and removes duplicate rows from the final result set.
- `UNION ALL` combines the results of two or more SELECT statements but includes all